

What Amber Will Look Like to Researchers

Let's first look at a motivating example:

Doctor Emmett Brown wants to run a simulation of a weather experiment in Hill Valley to get 95% confidence intervals around how much electricity is likely to be generated from a lightning strike. His experiment is mostly written in Python using NumPy to model the weather phenomenon. He has also written some custom C code that uses topographic information for the surrounding valley and the architectural composition of the buildings near the expected area of the strike. He runs the experiment on his computer and publishes the result. Dr. Brown shares his code on GitHub and his data set on his website. However, other researchers cannot replicate the results and so his work is discredited. Now, let's suppose that Dr. Brown had instead used amber. When compiling the code for his experiment, Dr. Brown simply types amber in his shell first and from then on, it transparently (and automatically) captures all the information about exactly how the environment is configured. Amber even notices the compiler flags and system libraries about the custom C code that Dr. Brown forgot to document. amber generates a record of what happened during the software compilation automatically, including the exact versions of libraries, floating point behaviour, etc. The actual run of the experiment proceeds in a similar manner. Likewise, Dr. Brown runs amber before running his experiment and so data about it is captured by amber. This ensures that if the experiment uses any external resources (such as extra libraries or hardware on the system), these are also accounted for. amber also replicates Dr. Brown's experiment in a cloud environment where other researchers can validate it and/or replicate this environment for themselves at will. As a result, Dr. Brown is commended for his great scientific findings!

To make amber work as described in the above example, we need to solve several difficult problems.

First, we need a means to capture a full and accurate picture of everything happening when code is executing. This includes information about the operating system, hardware, libraries used while executing, etc. The output of this step is an attestation which contains the environmental information, secure hashes of all used libraries, etc. To meet our goal of transparency, this needs to work across different environments without requiring a researcher to make any changes. Note, as in the example above, information is captured both while building the software and while executing the experiment.

Second, given an attestation, amber must collect and make sense of these. There are billions of software packages, libraries, etc. Understanding how to get the correct version of all software is a major challenge as is identifying which pieces of software are custom. This is much more difficult than just using a software version number, as different distributions may use the same version number and yet have different patches applied to the underlying code. This also runs into problems where people rebuilding software will usually not make a bit-for-bit identical version. Finally, we need to deal with software which is closed source (e.g., Windows), which is prevalent in some environments.

Finally, we need to replicate the experimental code to ensure we've correctly captured it. This involves replicating the environment and rebuilding the code in a public cloud provider. Note that this execution setup is all made public and so validation of reproducibility is now a trivial matter and does not require manual effort to validate that the environment can be replicated.